מבוא למדעי המחשב 67102 – סמסטר ב' 2024

תרגיל 4 – Battleships

להגשה בתאריך **06/06/2024** בשעה 22:00

משחק צוללות (Battleships)

בתרגיל זה אתן תממשו וריאציה של המשחק "צוללות". משחק זה מיועד לשני שחקנים, לכל שחקן יש לוח עליו הוא ממקם בתחילת המשחק צוללות, והוא מנסה להטביע את הצוללות בלוח של היריב ע"י שליחת פצצה אל קואורדינטה כלשהי בלוח המשחק. כל שחקן משחק בתורו (לסירוגין).

מטרת המשחק היא להטביע את כל הצוללות של היריב, והשחקן שעושה זאת ראשון מנצח.

ניתן לקרוא עוד על המשחק באתר ויקיפדיה: צוללות (משחק) – ויקיפדיה

בשונה מהמשחק הקיים, במימוש שלנו יתקיימו התנאים הבאים:

1. את הצוללות ניתן להציב במאונך בלבד.
2. לצוללות מותר להיות צמודות לצוללות אחרות.
3. המשחק ישוחק בין שחקן אנושי, לבין המחשב (שיקרא גם "היריב". בחירת הפעולות עבור המחשב מומשו עבורכם).

כמו במשחק הקיים – לצוללות אסור לחפוף אחת לשנייה (משמע, צוללת אחת לא יכולה לחלוק משבצת עם צוללת אחרת).

אנו ממליצים מאוד לקרוא את כל הוראות התרגיל המפורטות במסמך זה לפחות פעם אחת לפני שאתן ניגשות לפתרון!

בנוסף, הקפידו על כתיבת קוד קריא על פי חוקי ה- **Coding Style** שהוצגו בקורס!

מילוי הלוח:

בחלק זה עליכן לכתוב קוד שימקם באופן אינטראקטיבי (באמצעות קלט מהמשתמש) את הצוללות על לוח המשחק של השחקן האנושי.

כדי להקל על התהליך, מסופק לכן קובץ עזר - **helper.py** המכיל קבועים שונים ופונקציות עזר שונות.

עליכן לייבא (באמצעות import) את הקובץ לתוך הקובץ **battleship.py** כדי שתוכלו להשתמש בקבועים ובפונקציות המוגדרות בו.

שימו לב - עליכן להשתמש ישירות בקבועים המוגדרים בקובץ זה ולא להעתיק אותם לקוד שלכן!

מותר להגדיר קבועים נוספים אך עליהם להימצא בקובץ **battleship.py** בלבד ועליכן להשתמש בקבועים המסופקים לכן בקובץ **helper.py** במקומות הנדרשים לכך.

קובץ העזר - פירוט

הקבועים המוגדרים בקובץ **helper.py** (שעליכן להשתמש בהם - ישנם עוד קבועים שאין צורך להשתמש בהם) הם:

1. NUM_ROWS: קבוע שהוא מספר שלם המייצג את מספר השורות בלוח המשחק.
2. NUM_COLUMNS: קבוע שהוא מספר שלם המייצג את מספר העמודות בלוח המשחק.
3. SHIP_SIZES: קבוע מסוג tuple המכיל את גדלי הצוללות (שימו לב שיכולות להיות צוללות שונות באותו גודל).
4. WATER: קבוע המייצג תא בלוח המשחק שיש בו מים (= תא ללא צוללת = תא ריק. כלומר – תא שיש בו מים זה כמו לומר תא ריק.)
5. SHIP: קבוע המייצג תא בלוח שיש בו צוללת.
6. HIT_WATER: קבוע המייצג תא בלוח המשחק שהכיל מים ונפגע.
7. HIT_SHIP: קבוע המייצג תא בלוח המשחק שהכיל צוללת ונפגע.

הפונקציות המוגדרות בקובץ **helper.py** (שעליכן להשתמש בהן - ישנן עוד פונקציות שאין צורך להשתמש בהן) הן:

1. **print_board(board1, board2=None):**

פונקציה זו מקבלת לוח אחד או שני לוחות ומדפיסה אותם על המסך.
אם מסופק רק לוח אחד - הפונקציה מדפיסה רק אותו. אם מסופקים שני לוחות - הפונקציה תדפיס את שניהם.
שימו לב - הפונקציה תדפיס את הלוח במלואו כפי שהוא מתקבל בפונקציה.

אם ברצונכם שהלוח יודפס בצורה אחרת (למשל – להסתיר את הספינות שעוד לא נפגעו), יש להכין לוח מתאים.

2. **get_input(msg):**

פונקציה זו מקבלת הודעה להדפסה, מדפיסה הודעה זו ומחכה לקבלת קלט מהמשתמש - שימו לב להשתמש אך ורק בפונקציה זו כשאתן מבקשות קלט מהמשתמש בתרגיל!

3. **is_int(s):**

פונקציה זו מקבלת מחרוזת ובודקת שניתן להמיר אותה ל-int. אם ניתן - הפונקציה מחזירה True. אם לא ניתן - הפונקציה מחזירה False. (ניתן, במקום, להשתמש במתודה isdigit של מחרוזות)

4. **choose_ship_location(board, size, locations):**

פונקציה זו מקבלת לוח משחק, גודל של צוללת וקבוצת מיקומים אפשריים חוקיים (מהצורה - (**row_index,column_index**)) להצבת ראש הצוללת, ומחזירה מיקום אחד להצבת הצוללת.
מיקום חוקי יהיה כל מיקום בגבולות הלוח כך שכל חלקי הצוללת במים פנויים (כלומר מכיל את הערך WATER) ובגבולות הלוח.
הפונקציה יוצאת מנקודת הנחה שכל הערכים הנשלחים אליה חוקיים, שליחת מיקום לא חוקי (או בפורמט לא מתאים) יכול לגרום לתוצאות לא חוקיות או אפילו לקריסת התוכנית.

5. choose_torpedo_target(board, locations):

פונקציה זו מקבלת לוח (עם ספינות מוסתרות) וקבוצת מיקומים אפשריים (מהצורה –
(**(row_index,column_index)**) לשם שליחת פצצה (טורפדו) לצוללת של היריב ומחזירה מיקום לשליחת טורפדו.

<u>מימוש המשחק</u>

לוחות המשחק מיוצגים ע"י רשימה דו מימדית.

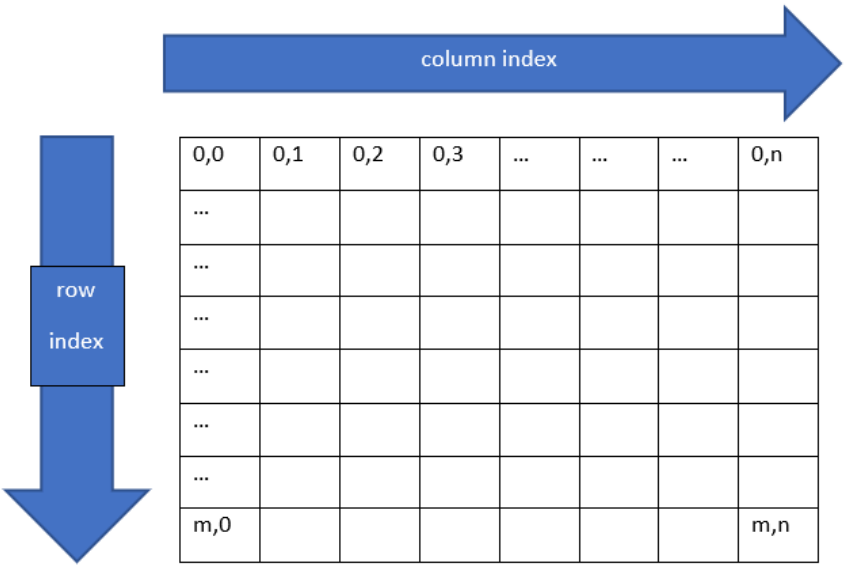
כל תא ברשימה דו מימדית (לאחר אתחול הרשימה) מייצג אלמנט בלשהו בלוח:

- מיקום של צוללת מיוצג ע"י הקבוע SHIP (מיוצג עם צבע חום בלוח).
- מיקום של פגיעה מיוצג ע"י הקבוע HIT_SHIP (מיוצג עם צבע אדום בלוח).
- מיקום של החטאה מיוצג ע"י הקבוע HIT_WATER (מיוצג עם צבע טורקיז בלוח).
- מיקום ריק (ללא צוללת) בלוח של השחקן (הלוח שלנו) או מיקום לא ידוע בלוח של היריב (המחשב) מיוצג ע"י הקבוע WATER (מיוצג עם צבע כחול בלוח).

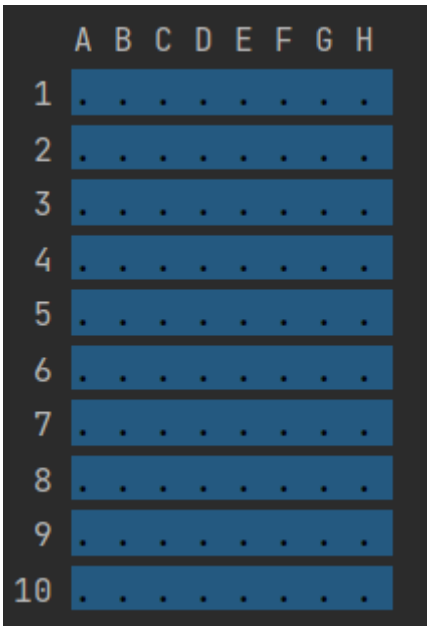
<u>הערות כלליות:</u>

1. בכל מקום בו מצויין "לוח המשחק" (או לוח) הכוונה היא לרשימה דו מימדית מלבנית המייצגת את לוח המשחק.
2. בכל מקום בו מצויין "מיקום" או "קואורדינטה" הכוונה היא ל-tuple המכיל זוג מספרים שלמים המהווים אינדקס של שורה ואינדקס של עמודה (בהתאמה) ברשימה דו מימדית המייצגת את הלוח:
   ((row_index,column_index
   (שימו לב שהאינדקס של השורה יבוא <u>לפני</u> האינדקס של העמודה).
   לדוגמא: (5,3) הוא התא הנמצא בשורה השישית ובעמודה הרביעית (מפני שאינדקסים בפייתון מתחילים מ-0, 5 הוא האינדקס הראשון ולכן מייצג את השורה ה1+5, ובאופן דומה 3 הוא האינדקס השני ולכן מייצג את העמודה ה1+3).
3. ייצוג הצבעים הוא פנימי לקובץ העזר, אין להשתמש/להוסיף/לשנות אותו.

4. דוגמא לייצוג של רשימה דו מימדית בעלת m שורות ו-n עמודות:



5. בעוד אנחנו עובדים עם מטריצה דו מימדית בעלת אינדקסים מספריים, לוח המשחק החשוף לשחקן מייצג מיקום ע"י אינדקס מספרי לשורות, ואינדקס לקסיקלי (מבוסס אותיות) עבור העמודות.



הוראות:

1. ממשו את הפונקציה:

init_board(rows, columns)

הפונקציה צריכה לקבל שני פרמטרים:

a. rows - מספר השורות בלוח.

b. columns - מספר העמודות בלוח.

על הפונקציה ליצור לוח ריק בגודל השורות כפול העמודות עבור השחקן או המחשב - לוח המלא כולו באותיות המסמלות מיקום ריק. הפונקציה תחזיר את הלוח.

שימו לב: הרשימה המוחזרת היא רשימה של רשימות בעלות ערכים זהים, אבל יש לוודא בעת היצירה שלה שהרשימות הן רשימות נפרדות.

2. ממשו את הפונקציה:

cell_loc(name)

הפונקציה מקבלת פרמטר יחיד בשם name, שמייצג קלט טקסטואלי מהמשתמש המתאר מיקום בלשהו בלוח ומחזירה tuple המתאר קואורדינטה מספרית.

הקלט שהשחקן מספק יהיה מהפורמט XN כאשר X מתארת עמודה בלשהי המיוצגת ע"י אות אנגלית גדולה, ו-N מתאר את מספר השורה (בשמתחילים ב- 1).

בעוד בקלט מהמשתמש אות קטנה ומספר יהיו ערכים חוקיים, הפונקציה הזו צריכה לקבל רק אות גדולה ומספר כפרמטר חוקי.

שימו לב! הקלט הפוך מייצוג קואורדינטה (בקלט הטקסטואלי מופיעה קודם העמודה ואז השורה, ואילו בייצוג של קואורדינטה מופיע קודם האינדקס של השורה ואז האינדקס של העמודה).

לדוגמא: אם המשתמש סיפק את הקלט "**A**2" הפונקציה תחזיר את הערך (1,0) שכן העמודה **A** היא העמודה הראשונה (בלומר באינדקס 0) ושורה 2 היא השורה באינדקס 1.

ניתן להניח שהערך שהתקבל הוא מסוג מחרוזת ובפורמט החוקי (בלומר תו ולאחריו מספר שלם).

לא ניתן להניח שהערך מייצג קואורדינטה בגבולות הלוח (שימו לב שהפונק' לא מקבלת לוח בקלט ולבן גם לא ניתן לבדוק זאת).

בנוסף, ניתן להניח שיהיו לכל היותר 26 עמודות.

בשביל לבצע את ההמרה בין הייצוג הטקסטואלי של אות לייצוג המספרי שלה ניתן להשתמש בפונקציה ord (ניתן לקרוא עוד על הפונקציה ord באן).

הפונקציה ord מקבלת מחרוזת באורך 1 המייצגת אות ומחזירה ערך מספרי עבור האות הזו לפי ייצוג ASCII.

למשל הקוד:

ord("A")

יחזיר את הערך 65, בעוד הקוד:

ord(“D”)

יחזיר את הערך 68. בשביל לבצע המרה חזרה מערך ASCII מספרי לייצוג טקסטואלי ניתן להשתמש בפונקציה chr (ניתן לקרוא עוד על הפונקציה chr באן).

למשל הקוד:

char(68)

יחזיר את הערך “D”.

3. ממשו את הפונקציה:

valid_ship(board, size, loc)

הפונקציה צריכה לקבל שלושה פרמטרים:

a. board – לוח משחק

b. size - גודל ספינה

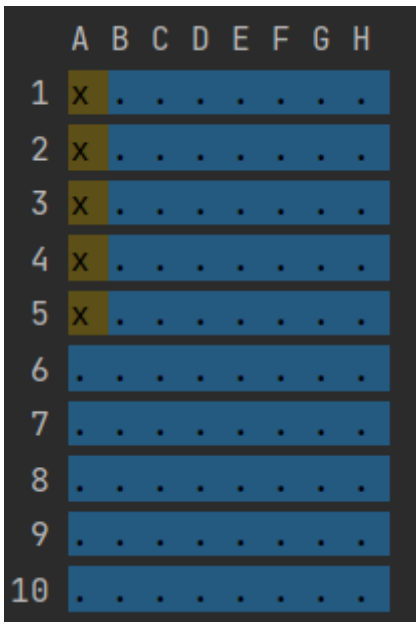
c. loc – משתנה מסוג tuple שמייצג מיקום בלוח (זוג מספרים שלמים) מהצורה – **(row_index,column_index)** באשר הראשון הוא האינדקס של השורה והשני של העמודה.

הפונקציה תבדוק האם צוללת בגודל size שמתחילה בקואורדינטה (מהצורה - **(row_index,column_index)**) יכולה להיכנס ללוח ללא התנגשות בקצות הלוח או בצוללת אחרת.

הפונקציה תחזיר True אם הצוללת יכולה להיכנס ללוח באופן חוקי, ו- False אחרת.

שימו לב שהצוללת מוכנסת במאונך, כאשר הקואורדינטה שהתקבלה מתארת את המקום הגבוה ביותר עבורה בלוח.

לדוגמא, ניתן להכניס צוללת בגודל 5 בקואורדינטה 1A באופן הבא:



שימו לב שלא נוכל להכניס עוד צוללת בקואורדינטה 2A כיוון שזו תתנגש עם הצוללת הראשונה שהוכנסה.
לא נוכל גם להכניס צוללת בגודל 5 בקואורדינטה 9A בלוח זה, כיוון שזו תגרום ליציאה מקצות הלוח.
כמו כן, שימו לב שפונקציה זו לא מבצעת שינויים בלוח, רק מבצעת בדיקה.

4. ממשו את הפונקציה:

create_player_board(rows, columns, ship_sizes)

הפונקציה צריכה לקבל שלושה פרמטרים:

a. rows – מספר השורות בלוח.
b. columns - מספר העמודות בלוח.
c. ship_sizes – רשימה או טופל (tuple) של מספרים שלמים שמתארים את אורכי הצוללות שיופיעו בלוח.

על הפונקציה לייצר לוח ריק חדש עבור השחקן, ולבקש מהמשתמש את הקואורדינטות בהן הוא מעוניין להניח צוללות. סדר גדלי הצוללות הוא לפי הסדר של ship_sizes.
כל עוד נותרו ספינות שצריך למקם, הפונקציה תדפיס את מצב הלוח הנוכחי (עם הפונקציה **print_board**) ולאחר מכן תבקש מהמשתמש (עם הפונקציה **get_input**) מיקום להנחת הספינה הבאה. יש להציג עם הבקשה הודעה מתאימה כדי שהשחקן יבין מה הוא אמור לעשות.
הקלט מהמשתמש יתקבל בעזרת הפונקציה **get_input** שמומשה עבורכם בקובץ **helper.py**, והפורמט בו הקלט צריך להתקבל הוא: **XN** כאשר X הוא אותר המייצגת תור ו-N הוא מספר שלם המייצג שורה.
הצוללת תמוקם על הלוח כאשר הקואורדינטה שניתנה היא הקואורדינטה העליונה ביותר (ראו דוגמא).
אם אכן ניתן למקם את הצוללת על הלוח במיקום זה, עדכנו את הלוח והמשיכו להשמת הצוללת הבא.
אם לא, הדפיסו שוב את הלוח, ובקשו קלט נוסף מהמשתמש עבור צוללת זו עם הודעה אינפורמטיבית (שיכולה להיות כל דבר, העיקר שתסמן שהקלט אינו חוקי).
בסוף הפונקציה יש להחזיר את לוח המשחק.

<u>הערות:</u>

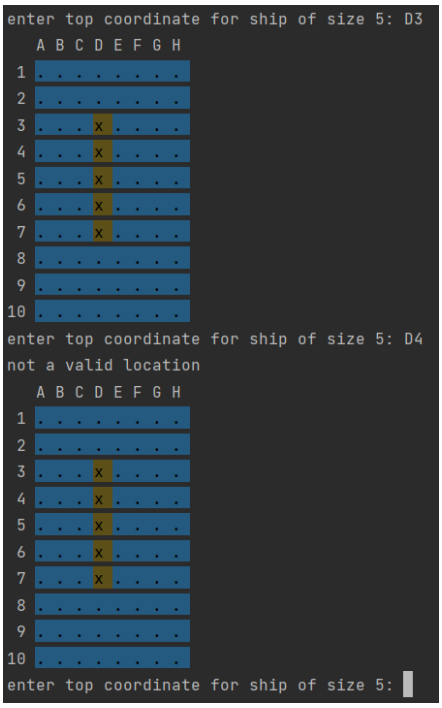
a. ניתן להניח שהקלט חוקי, ושניתן יהיה להכניס את הספינות ללוח.
b. המשחק אמנם לא יהיה מעניין, אך גם רשימה ריקה היא קלט חוקי.

<u>דוגמאות:</u>

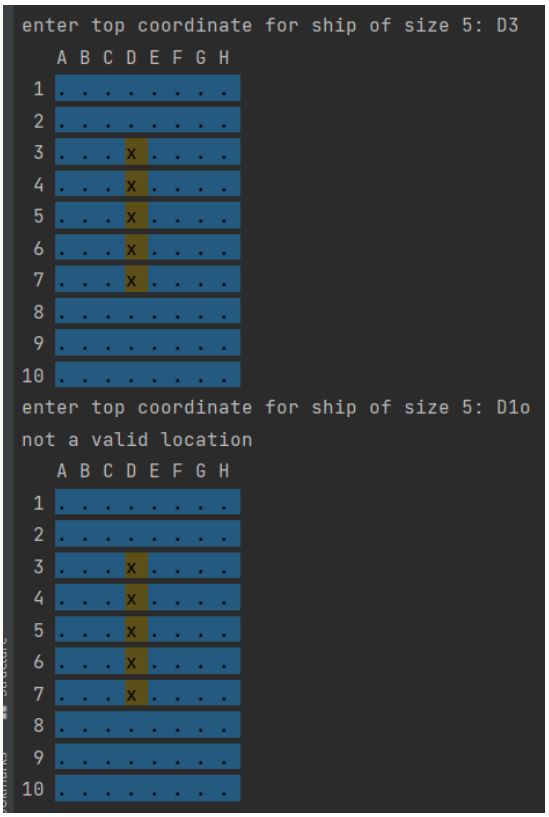
הכנסה חוקית:



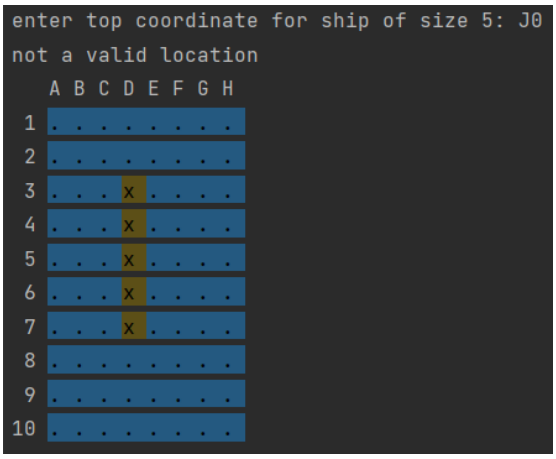
הכנסה לא חוקית (קלט הגורם להתנגשות עם צוללת אחרת, ועל כן הלוח לא השתנה):



הכנסה לא חוקית (קלט שלא נמצא בפורמט, על כן הלוח לא ישתנה):



הכנסה לא חוקית (מיקום לא חוקי):



5. ממשו את הפונקציה:

fire_torpedo(board, loc)

הפונקציה צריכה לקבל שני פרמטרים:

a. board – לוח המשחק.

b. loc – משתנה מסוג tuple שמייצג את המיקום בלוח (כאשר השורה מיוצגת ראשונה, כמו בשאר הtupleים בתרגיל). הפונקציה תבצע הפצצה על קואורדינטה מסוימת בלוח.

אם הפגיעה הייתה במים, הפונקציה תחליף בלוח את מיקום ההפצצה בלוח לקבוע המייצג מים חשופים.

אם הפגיעה הייתה בצוללת, הפונקציה תחליף בלוח את מיקום ההפצצה לקבוע המייצג פגיעה.

לאחר ביצוע השינוי הפונקציה תחזיר את הלוח.

אם התקבל מיקום שאינו חוקי, הפונקציה לא תבצע שינוי אך עדיין תחזיר את הלוח (ניתן להניח שהתקבל לוח חוקי).

6. ממשו את הפונקציה:

main()

הפונקציה תיצור את הלוחות שימוקמו בהן הצוללות עבור השחקן ועבור המחשב בעזרת הפונקציות שמימשתם בחלקים הקודמים (ופונקציות נוספות אם יש בכך צורך), ולאחר מכן תתחיל את המשחק עצמו. הפונקציה אחראית על הפעלת התורים במשחק. סיבוב אחד (מהלך שני תורים במשחק - אחד של השחקן ואחד של המחשב) מתבצע באופן הבא:

1. בתחילת יודפסו שני הלוחות בעזרת הפונקציה **print_board** שמומשה עבורכם בקובץ **helper.py**.

הפונקציה מקבלת שני לוחות כפרמטרים כאשר הלוח שמתקבל בפרמטר הראשון הוא הלוח שהשחקן יצר שיודפס באופן מלא (נראה את מיקומי הצוללות) והלוח שמתקבל בפרמטר השני הוא לוח מוסתר של המחשב, בו הספינות החבויות יודפסו כמים.

2. בקשת קלט - תודפס הודעה למשתמש שתבקש ממנו להכניס מיקום לשליחת פצצת טורפדו.

אם הקואורדינטה שהתקבלה תקינה והיא מסמנת תא בלוח שעדיין חבוי, נשנה את לוח המחשב בהתאם.

   a. השתמשו בפונקציה **get_input** שמומשה עבורכם בקובץ **helper.py**.

אם הקואורדינטה שהתקבלה אינה תקינה, נדפיס הודעה אינפורמטיבית (שיכולה להיות כל דבר, העיקר שתצביע על כך שהקלט לא תקין).

   b. ניתן לקבל אותיות קטנות כקלט תקין. בשביל להמיר אותיות קטנות לגדולות ניתן להשתמש בפונקציה upper של string (עוד מידע על הפונקציה upper בלינק.(

3. ביצוע הירי- ביצוע ירי הפצצה ע"י 2 הצדדים (המהלך של המחשב יוגרל בעזרת הפונקציה **choose_torpedo_target**) ועדכון הלוח.

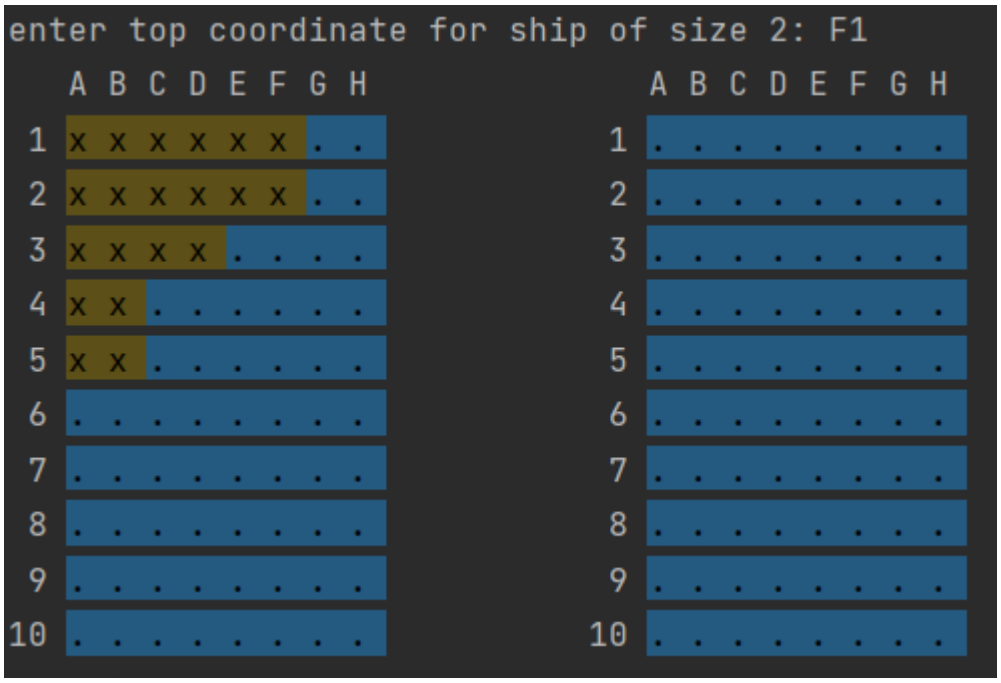
במקרה ולאחר פגיעה הושמד הצי של אחד השחקנים (או שניהם!), יודפסו שני הלוחות בצורה גלויה, ולאחר מכן שאלה אם לשחק שוב יחד עם הודעה על תוצאת המשחק שהסתיימה.

הקלט החוקי במקרה זה הוא "Y" או "N".

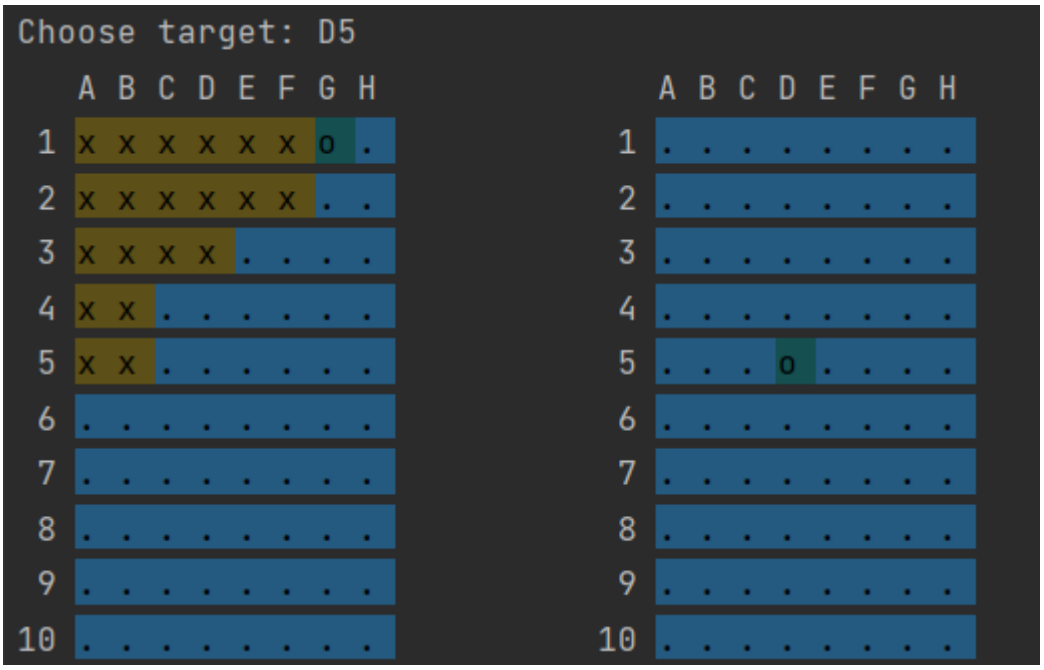
אם מתקבל קלט לא חוקי, יש להמשיך לשאול (עם הודעה מתאימה), אך ללא הדפסת הלוחות.

<u>דוגמאות:</u>

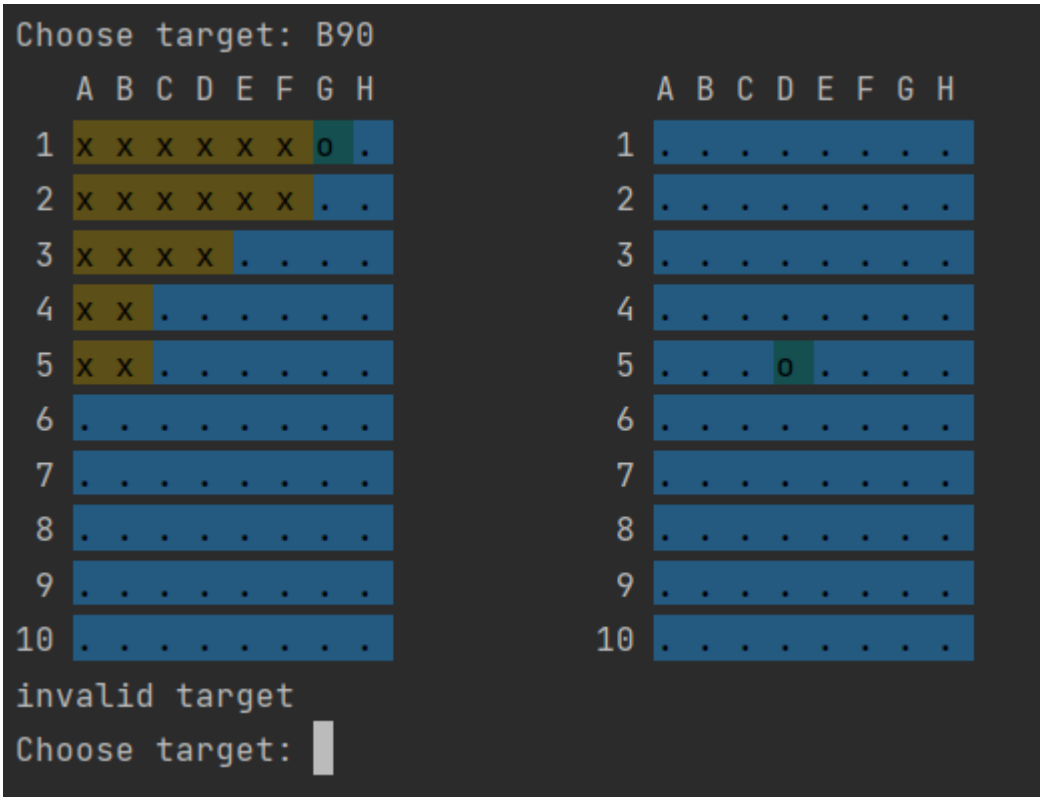
תחילת המשחק (הלוח של השחקן - שלנו - הוא הלוח השמאלי וניתן לראות בו את הצוללות. הלוח הימני הוא הלוח של היריב - המחשב - והצוללות בו לא נראות לנו):



דוגמא לקלט חוקי (הקלט שהכנסנו הוא 1B ולכן אנחנו מנסים לפגוע בצוללת שנמצאת בקואורדינטה (0,2). הפגיעה הצליחה כי הייתה שם צוללת של היריב):



דוגמא לקלט לא חוקי (הכנסנו בתור קלט את המחרוזת 90B שהוא קלט לא חוקי וקיבלנו הדפסה של invalid target):



הוראות הגשה

- עליכן להגיש את הקובץ ex4.zip (בלבד) בקישור ההגשה של תרגיל 4 דרך אתר הקורס (moodle).
  ההגשה היא עד המועד הנקוב בראש הקובץ.
- ex4.zip צריך לכלול את הקובץ הבא בלבד:
  - battleship.py.
- שימו לב שאין להגיש את הקובץ **helper.py**, ושהבדיקות יתבצעו עם קובץ עזר בעל ערכים שונים.

הנחיות כלליות בנוגע להגשה

- הנכם רשאים להגיש תרגילים דרך מערכת ההגשות באתר הקורס מספר רב של פעמים. ההגשה האחרונה בלבד היא זו שקובעת ושתיבדק.
- לאחר הגשת התרגיל, ניתן ומומלץ להוריד את התרגיל המוגש ולוודא כי הקבצים המוגשים הם אלו שהתכוונתם להגיש וכי הקוד עובד על פי ציפיותיכם.
  - באחריותכם לוודא כי – **PDF** הבדיקות נראה כמו שצריך.
- קראו היטב את קובץ נהלי הקורס לגבי הנחיות נוספות להגשת התרגילים.

צירפנו את הפלט הצפוי של בדיקת ה-presubmit של הפונקציה main עם הערות לעיונכם. שימו לב – הפונקציה לא אמורה להחזיר ערך כלשהו.

```
[[[5, 5, 5], [5, 5, 5]]]              # הצגת הלוח לפני הצבת הספינה הראשונה
I[A1]                                 # בקשת קלט. המחרוזת בתוך הסוגריים היא המחרוזת המוחזרת
[[[6, 5, 5], [6, 5, 5]]]              # הלוח לפני הצבת הספינה השניה. שתי הספינות הן באורך 2
I[B1]#    board             size     sorted(locations)
('S', [[5, 5, 5], [5, 5, 5]], 2, [(0, 0), (0, 1), (0, 2)]) # בקשת מיקום לספינת מחשב
('S', [[6, 5, 5], [6, 5, 5]], 2, [(0, 1), (0, 2)])        # (ניתן למקם קודם את ספינות המחשב.)
[[[6, 6, 5], [6, 6, 5]], [[5, 5, 5], [5, 5, 5]]]                        # הדפסה לפני התור הראשון
I[A1]
('T', [[5, 5, 5], [5, 5, 5]], [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2)])
[[[8, 6, 5], [6, 6, 5]], [[8, 5, 5], [5, 5, 5]]]                        # הדפסה לפני התור השני
I[B1]#     board                    sorted(locations)
('T', [[8, 5, 5], [5, 5, 5]], [(0, 1), (0, 2), (1, 0), (1, 1), (1, 2)])   # מחשב בוחר מטרה
[[[8, 8, 5], [6, 6, 5]], [[8, 8, 5], [5, 5, 5]]]
I[C1]
('T', [[8, 8, 5], [5, 5, 5]], [(0, 2), (1, 0), (1, 1), (1, 2)])
[[[8, 8, 7], [6, 6, 5]], [[8, 8, 7], [5, 5, 5]]]
I[A2]
('T', [[8, 8, 7], [5, 5, 5]], [(1, 0), (1, 1), (1, 2)])
[[[8, 8, 7], [8, 6, 5]], [[8, 8, 7], [8, 5, 5]]]
I[B2]
('T', [[8, 8, 7], [8, 5, 5]], [(1, 1), (1, 2)])
[[[8, 8, 7], [8, 8, 5]], [[8, 8, 7], [8, 8, 5]]]              # הדפסה שבה לוח המחשב גלוי לאחר סיום המשחק
I[N]                              # (התשובה היא לא) האם לשחק שוב
# התו האחרון הוא תו סיום שורה.
```

בהצלחה!